

ENG-001

Opus X Reference Data Model (Supabase / Postgres)

DRAFT — EXECUTION PHASE

DOCUMENT	ENG-001
SERIES	Engineering
VERSION	0.1
STATUS	Draft — Execution Phase
DATE	2026-07-10
LANGUAGE	English

NOTE — Position of this document. ENG-001 is the first Engineering artifact of the execution phase (opened by ARCH-002 §7). It translates the normative corpus into a concrete reference data model for Opus X on Supabase/Postgres: it projects the object model (WSP-001), the canonicalization contract (SCHEMA-001), the trust flow (ARCH-001), and the services (OPUS-400) into tables, constraints, and access rules. It is a *faithful projection*, not a redesign: every schema decision traces to a corpus clause, and the protocol's invariants are enforced *structurally* (append-only facts, physical fact/interpretation separation, subject-controlled access). It is a reference for Code (the next role in the cycle), and it introduces no new protocol or product concepts. This is a Draft.

Version History

Version	Date	Status	Summary of Changes
0.1	2026-07-10	Draft — Execution Phase	Initial publication. Reference schema for Subjects, Issuers, Frameworks, Skills, Evidence, Relations, Certifications, Interpretations, and the Passport view, with invariants enforced structurally and the ARCH-001 ingestion flow.

Table of Contents

1. Introduction.....	4
1.1 Purpose.....	4
1.2 Scope.....	4
1.3 Normativity	4
1.4 Prerequisites	4
2. Design Principles (Traced to the Corpus).....	4
3. The Schema at a Glance	5
4. Identity and Common Columns	6
5. The Fact Store (Append-Only)	6
6. Semantics: Frameworks and Skills	7
7. Interpretations (Physically Separated)	8
8. The Passport (A Generated View).....	8
9. The Ingestion Flow (ARCH-001)	9
10. Access Control: Subject Control as RLS	10
11. What Is Locked vs. What Is Implementation Detail	10
12. Handoff to Code	11
13. Summary.....	11
References	12
Informative References.....	12

1. Introduction

1.1 Purpose

This document turns the specification into a technical data model — the first concrete step of building Opus X. It defines a reference Supabase/Postgres schema that faithfully realizes the protocol's objects and invariants, so that Code can implement a Registry, verification, interpretation, and Passport generation that are correct by construction. It is the bridge from the normative corpus to running software, and to the Commando OS laboratory.

1.2 Scope

The reference data model: tables, keys, constraints, access rules, and the ingestion flow, for the core objects. Out of scope: the API surface (a later Engineering artifact), the interpretation *methods* (platform artifacts, MODEL/OPUS-300), the frontend (Code/Product), and the wire encoding for exchange (SCHEMA-003). This is a *reference* model; a concrete implementation may differ in detail where §11 permits.

1.3 Normativity

Informative as an Engineering reference. It does not create protocol requirements; it *implements* them. Where it says "must," it is restating a corpus normative rule (cited), not creating a new one. The invariants it enforces are owned by WSP-001 §14, FOUNDATION-000, and ARCH-001.

1.4 Prerequisites

WSP-001 (the objects and their invariants), SCHEMA-001 (canonicalization), WSP-002 (identifiers), ARCH-001 (the trust flow and role separation), OPUS-400 (the services this data model underlies), and ARCH-002 (the operating method this artifact is produced under).

2. Design Principles (Traced to the Corpus)

Every principle of this data model is a projection of a corpus rule. This traceability is the QA discipline (ARCH-002 §4): no schema decision is arbitrary.

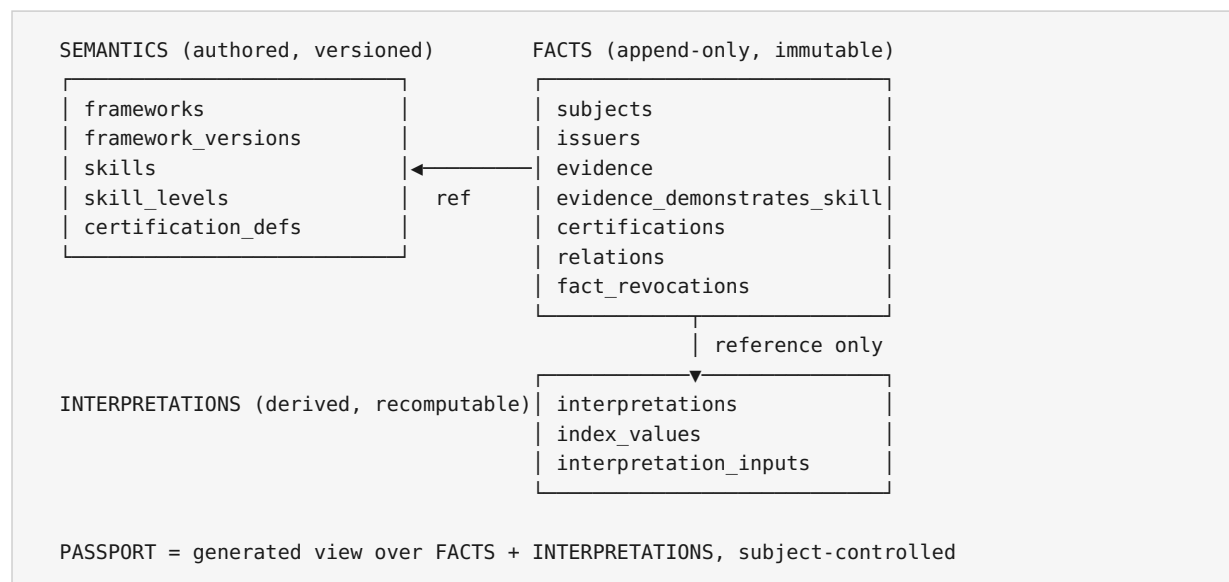
- **Facts are append-only; never mutated** (WSP-001 §14.3, §3.4). Fact tables permit INSERT only — no UPDATE, no DELETE. Corrections and revocations are new fact rows referencing the old.
- **The Registry records; it does not interpret** (WSP-001 §13.3; FOUNDATION-000 Annex C). Fact tables carry no computed/derived columns; interpretations live in physically separate tables (§7).

- **Interpretations never become facts** (WSP-001 §14.2). The separation is physical and one-directional: interpretation tables reference facts; no process writes an interpretation into a fact table.
- **Every meaning-bearing object carries its Framework version** (WSP-001 §3.6). `framework_id` + `framework_version` columns are mandatory wherever meaning depends on domain semantics, so meaning is recoverable in perpetuity.
- **Every fact is attributed and dated** (WSP-001 §3.2, §3.3). Fact tables require `issuer_id` and an immutable `recorded_at`.
- **Evidence is produced by issuers; trust is operated by the platform** (ARCH-001 §5). The schema records *who produced* each fact (`issuer_id`) separately from *the platform's verification* of it (§9), so no producer self-attributes trust.
- **The subject controls disclosure** (WSP-001 §12.3; WSP-005). Access is governed by row-level security keyed to the subject (§10).
- **Identity is stable and never reused** (WSP-001 §3.1; WSP-002 §5.1). Identifiers are protocol identifiers, unique forever, never recycled.

These principles are locked (§11); the specific mechanics that realize them are implementation detail that Code and later revisions may refine.

3. The Schema at a Glance

The schema divides into three physically separated zones, mirroring the object model's fact/interpretation boundary (WSP-001 §2):



The three zones are the schema-level enforcement of the protocol's defining discipline: semantics are referenced, facts are immutable and append-only, interpretations are derived and physically separate, and the Passport is a generated view — never a stored, authored document (WSP-001 §12.2).

4. Identity and Common Columns

Every object carries the common envelope (SCHEMA-001 §3; WSP-001 §3):

- **id** — the protocol identifier (WSP-002 §3), stored in canonical written form. Primary key. A protocol identifier, globally unambiguous, never reused (WSP-001 §3.1).
- **protocol_version** — the protocol version under which the object was recorded, so its schema and canonicalization are always recoverable (SCHEMA-001 §6.4).
- **recorded_at** — an immutable timestamp of when the fact entered the store (WSP-001 §3.3). Set on insert, never changed.
- **canonical_hash** — the hash of the object's canonical form (SCHEMA-001 §5), the basis of verification and integrity. Computed at ingestion, stored, never recomputed into a different value for the same object.

For fact objects, additionally: - **issuer_id** — the accountable creator (WSP-001 §3.2, §5.2). Foreign key to issuers.

A note on identifiers: the protocol identifier (id) is the object's canonical identity and is used for all cross-object references and exchange. A database may carry an internal surrogate key for storage efficiency, but the protocol id is authoritative and is what appears in relations, exports, and signatures (WSP-002 §3.4) — the surrogate is never exposed as identity.

5. The Fact Store (Append-Only)

The fact tables are the Registry (WSP-001 §13). They are **append-only by construction**: the reference schema grants INSERT and SELECT only, and revokes UPDATE and DELETE, on every fact table — so immutability is enforced by the database, not merely by convention (WSP-001 §14.3).

- **subjects** — the recorded existence of a Subject (the professional; WSP-001 §4). Carries identity; the professional's controllable profile. A subject's *facts* are their evidence, certifications, and relations — the subject row is their anchor.
- **issuers** — the recorded existence of an Issuer: an accountable entity that produces evidence or certifications (WSP-001 §5; ARCH-001 §4). Carries identity and accountability metadata. Partner platforms (Commando OS, AI schools), employers, and the reference implementations are issuers.
- **evidence** — a dated, attributable, verifiable record that a skill was exercised or assessed (WSP-001 §8). Columns: envelope + subject_id, issuer_id, occurred_at, context, provenance, evidence_kind, is_declaration (WSP-001 §8.5 — declarations distinguishable from evidence), framework_id, framework_version. Append-only.
- **evidence_demonstrates_skill** — the *demonstrates* relation from evidence to the skill(s) it concerns, at a claimed level (WSP-001 §8.2, §9.2). Kept as its own table so one evidence item may demonstrate several skills.

- **certifications** — a formal attestation by an issuer under a framework that requirements were met at a time (WSP-001 §10). Columns: envelope + subject_id, issuer_id, issued_at, framework_id, framework_version, and the requirements met (as skill-at-level rows). Attests attainment *at issuance*; never asserts continued attainment (WSP-001 §10.3) — currency is an interpretation (§7), not a column on the certification.
- **relations** — typed, dated, attributable connections between objects (WSP-001 §9). Columns: envelope + from_id, to_id, relation_type (constrained to the protocol's defined types only — WSP-001 §9.2, §9.5), issuer_id. Referential integrity enforced: a relation to a nonexistent object is rejected (WSP-001 §9.3).
- **fact_revocations** — a revocation is itself a new fact referencing the revoked fact (WSP-001 §8.4, §10.4, §14.3). Revocation never deletes; it records a new row. The revoked fact remains; its revoked *status* is derived by reading whether a revocation references it.

The relation_type column is constrained (e.g. by a Postgres CHECK or enum) to exactly the protocol's relation types — *demonstrates*, *supports*, *requires*, *attests*, *supersedes* (WSP-001 §9.2) — so applications cannot invent relation types (WSP-001 §9.5). New types come only from protocol evolution (a schema migration in step with a protocol version, SCHEMA-001 §6.2).

6. Semantics: Frameworks and Skills

The semantics tables hold the authored, versioned domain meaning (WSP-001 §6–§7). They are not facts and not interpretations; they are the referenced definitions.

- **frameworks** — a published professional-domain definition (WSP-001 §6). Identity and metadata.
- **framework_versions** — each version of a framework, preserved permanently so any fact recorded under a version stays interpretable forever (WSP-001 §6.3, §3.6; FRAMEWORK-003 §9). *Versions are never deleted or altered* — the same append-only discipline, for the same reason: a past fact's meaning must remain recoverable.
- **skills** — a nameable capability defined within a framework version (WSP-001 §7). Identified in the framework's namespace; carries the framework version that gives it meaning.
- **skill_levels** — the competence levels and criteria for a skill, which are *framework content* (WSP-001 §7.3) — the protocol stores them as referenced definitions, and does not itself define what a level means.
- **certification_defs** — the definition of a certification's requirements within a framework (referenced by certifications facts).

Crucially, fact tables reference a specific framework_version, never just a framework — so a piece of evidence or a certification is always interpreted under the exact version in force when it was recorded, and a later framework revision never silently changes what

a past fact meant (WSP-001 §3.6; FRAMEWORK-003 §9). This is how the schema keeps meaning stable across a domain — especially vital for a fast-moving domain like AI (FRAMEWORK-AI-001 §7).

7. Interpretations (Physically Separated)

Interpretations — competence levels read from evidence, the indices, freshness — live in their own tables, physically separated from facts (WSP-001 §11, §14.2):

- **interpretations** — a derived read over facts: what the platform infers (a competence level, a readiness, a freshness). Columns: `subject_id`, `interpretation_kind`, `value`, `method_version` (the versioned interpretation method that produced it — WSP-001 §11.5), `computed_at`, and references to the facts it rests on.
- **index_values** — the computed indices (Skills, Trust, Readiness, Growth, Coverage, Freshness, Expertise, Consistency; OPUS-300) as interpretations, each with its `method_version` and `computed_at`.
- **interpretation_inputs** — the explicit links from each interpretation to the facts it was derived from — the stored "explanation is a path" (MODEL-004 §4.1; OPUS-300), so every index opens to its evidence (PRODUCT-500 §4).

Three structural guarantees enforce the invariants:

- **Separation is physical and one-directional.** Interpretation tables reference fact tables (via `interpretation_inputs`); no foreign key or process runs the other way. Nothing writes an interpretation into a fact table (WSP-001 §14.2).
- **Interpretations are recomputable, never authoritative over facts.** An interpretation row can be recomputed and replaced as methods improve (`method_version` changes); the facts it read are untouched. Recomputation is not mutation of facts (MODEL-002 §4.4).
- **Every interpretation carries its method version and inputs.** So it is always explainable and reproducible (MODEL-002 §4.2, §4.5) — no index is an unexaminable number (PRODUCT-500 §4).

This physical separation is the schema-level enforcement of the fact/interpretation boundary — the protocol's defining discipline (WSP-001 §14). The Registry (fact tables) records and never computes; Trust Intelligence (interpretation tables) computes and never mutates facts.

8. The Passport (A Generated View)

The Passport is **not a table**. It is a generated view over a subject's facts and interpretations (WSP-001 §12.2 — "generated, not authored; nothing not derivable from recorded facts and their interpretations"). This is a critical structural decision: storing

the Passport as an authored document would violate §12.2 and risk it drifting from the facts. Instead:

- **The Passport is a query/view** composing a subject's evidence, certifications, relations (facts) and their interpretations (indices, levels, freshness), with facts and interpretations distinguished on its face (WSP-001 §12.5; PRODUCT-100 §6).
- **It is always current with the facts**, because it is derived from them at read time (or materialized and refreshed on fact/interpretation change) — it cannot drift from the truth (PRODUCT-100 §5).
- **It is subject-controlled** at the view level: what a given relying party sees is filtered by the subject's disclosure choices (§10; WSP-005 §4).
- **It references framework versions** so its facts are interpreted under the correct version (WSP-001 §12.5).

Generating rather than storing the Passport is how the schema guarantees the Passport is always a faithful, current, non-authored representation — the product's core property (PRODUCT-100) enforced at the data layer.

9. The Ingestion Flow (ARCH-001)

The flow by which evidence enters and the Passport updates realizes ARCH-001 §6 — *Evidence Is Produced. Trust Is Verified.* — as concrete steps:

1. **A certified issuer produces evidence** (a partner platform, e.g. Commando OS) and transmits it to Opus X via the protocol API (ARCH-001 §6 step 2-3). The evidence carries its issuer, subject, framework version, and canonical form.
2. **Opus X verifies** (ARCH-001 §6 step 4): it checks WSP conformance (schema valid per SCHEMA-001), verifies authenticity and integrity (canonical_hash, signature — SCHEMA-003), confirms the issuer is a recognized, accountable issuer, and confirms referential integrity (the framework version and skills exist). *Verification is the platform's, not the producer's* (ARCH-001 §5).
3. **On success, the evidence is recorded** as an append-only fact (evidence + evidence_demonstrates_skill) — the Registry writes a fact, and only a fact (WSP-001 §13).
4. **Interpretations are (re)computed** (ARCH-001 §6 step 4): the affected indices and levels are recomputed by the versioned methods and written to the interpretation tables, with their inputs recorded. The Registry did not compute; Trust Intelligence did (WSP-001 §13.3).
5. **The Passport reflects the change** (ARCH-001 §6 step 5): because the Passport is a generated view (§8), it now shows the new evidence and updated interpretations, certified and signed by Opus X, consultable by the subject and authorized parties.

The schema enforces the separation the flow depends on: the issuer writes nothing directly to the fact store (they submit; Opus X verifies and records), and no issuer writes to the interpretation tables at all (trust is operated by the platform). *A producer*

cannot self-attribute trust because a producer has no write path to the verified fact store or the interpretations — ARCH-001 §5 made structural.

10. Access Control: Subject Control as RLS

Subject control (WSP-001 §12.3; WSP-005 §4) is enforced by Postgres row-level security (RLS), so it is a property of the data layer, not just the application:

- **The subject owns their facts.** RLS policies grant a subject full read of their own facts and interpretations, and control over disclosure — the subject is the owning principal for their rows.
- **Disclosure is explicit and revocable.** A disclosures table records what a subject has shared, with whom, and for what purpose (WSP-005 §4.2–§4.4); RLS grants a relying party read access to a subject's Passport view *only* through an active, non-revoked disclosure. Revoking = marking the disclosure inactive (a new fact), which immediately ends access (WSP-005 §4.4).
- **No leakage.** RLS ensures a relying party sees only the disclosed subset, never the rest of the Passport or the underlying undisclosed facts (WSP-005 §6; PRODUCT-700 §3).
- **Issuers write via verified ingestion only.** Issuers have no direct write to fact tables; they submit through the verification path (§9), which the service layer performs with the platform's authority, not the issuer's.

Enforcing subject control as RLS means the professional's ownership is guaranteed by the database itself — even a bug in the application layer cannot expose what the subject did not disclose. Control felt at the product surface (PRODUCT-100 §7) is control enforced at the data layer.

11. What Is Locked vs. What Is Implementation Detail

Applying the QA discipline (ARCH-002 §4), the document is explicit about the boundary of its commitments.

Locked (these realize invariants; they may not be traded away): - Fact tables are append-only; no UPDATE/DELETE (WSP-001 §14.3). - Facts and interpretations are physically separated, one-directional (WSP-001 §14.2). - Every meaning-bearing object carries its framework version (WSP-001 §3.6). - Every fact is attributed and dated (WSP-001 §3.2–§3.3). - Issuers cannot write to the verified fact store or interpretations directly; the platform verifies and records (ARCH-001 §5). - The Passport is generated, not stored as an authored document (WSP-001 §12.2). - Subject control governs disclosure (WSP-001 §12.3).

Implementation detail (Code and later revisions may refine): - Exact table and column names, types, and indexing. - Whether the Passport view is computed on read or materialized and refreshed. - The surrogate-key strategy, partitioning, and performance

tuning. - The concrete RLS policy expressions. - The choice of Postgres/Supabase features used to enforce append-only (triggers, revoked grants, or both).

This boundary is what lets Code build freely while guaranteeing the build cannot erode the architecture: the locked items are the invariants made structural; everything else is engineering latitude.

12. Handoff to Code

Per the operating cycle (ARCH-002 §5), this Engineering artifact prepares the work for Code. The concrete next steps this model hands off:

- **Migrations** creating the three zones (semantics, facts, interpretations), with append-only enforcement on fact tables and RLS for subject control.
- **The ingestion service** implementing the ARCH-001 verify-then-record flow (§9), which is the only write path to verified facts.
- **The interpretation service** computing indices from facts with recorded inputs and method versions (§7), the only writer to interpretation tables.
- **The Passport view** generating the subject-controlled, fact/interpretation-distinguished representation (§8).
- **QA review** against §11's locked list before any delivery (ARCH-002 §4 — QA gates delivery).

Commando OS, as the laboratory (ARCH-002 §7; PRODUCT-000 §7), is where this model first meets real evidence, and where lessons flow back to refine both this model and the corpus.

13. Summary

This is the first Engineering artifact of the execution phase: a reference Supabase/Postgres data model for Opus X that faithfully projects the normative corpus into buildable structure. It divides into three physically separated zones — authored semantics, append-only facts, and derived interpretations — mirroring the protocol's defining fact/interpretation boundary, with the Passport as a generated view rather than a stored document. Every design decision traces to a corpus clause, and the invariants are enforced *structurally*: facts are append-only by revoked grants, interpretations are physically separate and one-directional, framework versions are carried everywhere, issuers have no direct write path so they cannot self-attribute trust, and subject control is enforced as row-level security. The ingestion flow realizes ARCH-001's *Evidence Is Produced. Trust Is Verified.* as concrete steps: issuers submit, the platform verifies and records facts, Trust Intelligence recomputes interpretations, and the Passport view reflects the change. The document is explicit about what is locked (the invariants made structural) versus what is implementation latitude, and it hands off clean, well-bounded work to Code — with Commando OS as the laboratory where the model meets reality. This is how the architecture becomes software without eroding.

References

Informative References

- WSP-001 — Core Objects and Semantics (§3, §4, §5, §6, §7, §8, §9, §10, §11, §12, §13, §14); SCHEMA-001 — Object Schemas (§3, §5, §6); WSP-002 — Identifiers and Versioning (§3, §5); SCHEMA-003 — Exchange Envelopes and Signatures
- ARCH-001 — Evidence Is Produced. Trust Is Verified. (§4, §5, §6); ARCH-002 — The Opus X Operating Method (§4, §5, §7); OPUS-400 — The Supporting Infrastructure; OPUS-300 — Professional Trust Intelligence
- MODEL-002 — Trust and Interpretation Model (§4); MODEL-004 — Professional Graph Model (§4.1); WSP-005 — Passport and Disclosure (§4, §6); FRAMEWORK-003 — Framework Lifecycle (§9); FRAMEWORK-AI-001 — AI Domain Taxonomy (§7)
- PRODUCT-100 — The Professional Passport (§5, §6, §7); PRODUCT-500 — The Trust Intelligence Experience (§4); PRODUCT-700 — The Sharing & Verification Experience (§3); FOUNDATION-000 — The Trust Manifesto